

Aula 2 — Playwright Avançado

"Selenium é padrão? Devo migrar?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/10/2026

Recap da Aula 1

- ISO 25010, Pirâmide vs Trophy vs Honeycomb
- POM vs Screenplay
- Flakiness e mitigação
- Setup Playwright + Maestro

Quiz Fundamentos respondido?

Agenda da aula (3h30)

- **90min** — Playwright avançado: arquitetura interna, locators, network mocking, visual regression (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: Playwright SPA (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Objetivos M2

- **Compreender** arquitetura interna do Playwright (CDP, contexts)
- **Aplicar** locators recomendados (role, text, test-id)
- **Implementar** network mocking + HAR replay
- **Configurar** storageState para auth reuse
- **Aplicar** visual regression nativo (`toHaveScreenshot`)
- **Configurar** sharding 4-way em CI

Por que Playwright?

Em 2026, Playwright tomou espaço de Selenium e Cypress em times sérios.

Razões técnicas:

1. **Chrome DevTools Protocol** (não WebDriver) — controle granular
2. **Browser contexts isolados** — paralelismo limpo
3. **Auto-waiting nativo** — sem sleeps
4. **Locators recomendados** — role, text, test-id
5. **Trace viewer** — debug determinístico
6. **Cross-browser** — Chromium, Firefox, WebKit
7. **Visual regression nativo** — `toHaveScreenshot`

Arquitetura — CDP vs WebDriver

Selenium (WebDriver):

- Protocolo HTTP REST padronizado W3C
- Compatível com infinitos browsers
- Cada comando = chamada HTTP

Playwright (CDP):

- Protocolo de baixo nível (Chrome DevTools)
- Conexão WebSocket persistente
- Acesso granular a network, runtime, performance

Diferença prática: Playwright é tipicamente 2–4x mais rápido em mesma suíte.

Browser context — isolamento

```
const browser = await chromium.launch();

// Cada context é isolado: cookies, storage, history
const ctx1 = await browser.newContext({ locale: 'pt-BR' });
const ctx2 = await browser.newContext({ locale: 'en-US' });

const page1 = await ctx1.newPage();
const page2 = await ctx2.newPage();
```

| 1 browser, N contexts, M pages — paralelismo natural.

Locators recomendados

```
// ✅ Bom — role + nome
page.getByRole('button', { name: 'Entrar' });
page.getByRole('link', { name: 'Sobre nós' });

// ✅ Bom — texto visível
page.getByText('Welcome back');

// ✅ Bom — test-id (estável)
page.getByTestId('submit-button');

// ✅ Bom — placeholder, label
page.getByPlaceholder('Email');
page.getByLabel('Senha');

// ❌ Ruim — frágil
page.locator('div.container > .card:nth-child(2) span.title');
```


Auto-waiting — sem sleeps

```
// Playwright aguarda elemento estar:  
// - Anexado ao DOM  
// - Visível  
// - Estável (não animando)  
// - Habilitado  
// - Receivable (não coberto)  
  
await page.getByRole('button', { name: 'Entrar' }).click();  
// Já aguardou tudo automaticamente  
  
// Web-first assertions também aguardam  
await expect(page.getByText('Welcome')).toBeVisible({ timeout: 5000 });
```

Se ainda precisa de `waitForTimeout`, está fazendo errado.

Network mocking — `route()`

```
test('handles API error gracefully', async ({ page }) => {  
  // Intercepta antes da request  
  await page.route('**/api/checkout', async (route) => {  
    await route.fulfill({  
      status: 500,  
      contentType: 'application/json',  
      body: JSON.stringify({ error: 'Internal Server Error' }),  
    });  
  });  
  
  await page.goto('/checkout');  
  await page.getByRole('button', { name: 'Pagar' }).click();  
  
  await expect(page.getByText('Erro. Tente novamente.')).toBeVisible();  
});
```

HAR replay

Captura tráfego real e replay determinístico em testes.

```
// Capturar
const context = await browser.newContext({
  recordHar: { path: 'session.har' },
});

// Replay
await page.routeFromHAR('session.har', {
  url: '**/api/**',
  notFound: 'fallback',
});
```

Útil para testar fluxos complexos sem depender de backend rodando.

storageState — auth reuse

Sem reuso: cada teste faz login (3s × 200 testes = 10min só de login).

```
// global-setup.ts — roda 1 vez
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('/login');
  await page.fill('[name=email]', 'user@email.com');
  await page.fill('[name=password]', 'secret');
  await page.click('button[type=submit]');
  await page.waitForURL('/dashboard');

  await page.context().storageState({ path: 'auth.json' });
  await browser.close();
}
```

```
// playwright.config.ts
export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  use: { storageState: 'auth.json' },
});
```

Visual regression nativo

```
test('home page visual', async ({ page }) => {  
  await page.goto('/');  
  await expect(page).toHaveScreenshot('home.png', {  
    maxDiffPixels: 100,  
  });  
});  
  
// Tirar baseline na primeira execução:  
// npx playwright test --update-snapshots
```

Para múltiplos viewports, configure projects no `playwright.config.ts`.

Trace viewer — debug determinístico

```
// playwright.config.ts
use: {
  trace: 'retain-on-failure', // captura trace em failures
}
```

```
# Abrir trace de execução
npx playwright show-trace trace.zip
```

UI mostra:

- Timeline de cada ação
- DOM snapshots a cada step
- Network requests
- Console logs
- Source code do teste

Component Testing

Alternativa a Cypress CT.

```
// LoginForm.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import LoginForm from './LoginForm';

test('disables submit when email invalid', async ({ mount }) => {
  const component = await mount(<LoginForm />);
  await component.locator('[name=email]').fill('invalid');
  await expect(component.locator('button[type=submit]')).toBeDisabled();
});
```

Roda componente isolado em browser real, sem app inteiro. Tempo médio entre unit e E2E.

Sharding 4-way em CI

```
# .github/workflows/playwright.yml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      shard: [1/4, 2/4, 3/4, 4/4]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: blob-report-${{ matrix.shard }}
          path: blob-report
```


Blob reports — agregação

```
merge-reports:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
    - run: npx playwright merge-reports --reporter html ./blob-report
    - uses: actions/upload-artifact@v4
      with: { name: html-report, path: playwright-report }
```

Resultado: 1 relatório consolidado de 4 shards paralelos.

Lab — suíte E2E completa (90min)

Roteiro

1. Clone starter `modulo-02` (5min)
2. Criar 5 testes E2E para SPA exemplo (TodoMVC) (20min)
3. Aplicar Page Object Model (15min)
4. Network mocking de erro 500 (10min)
5. `storageState` (auth reuse) (10min)
6. Visual regression em 3 viewports (10min)
7. Sharding em GitHub Actions (15min)
8. Trace habilitado (5min)

Pitfalls comuns

- ✗ `waitForTimeout` em vez de auto-waiting
- ✗ Locators frágeis dependentes de hierarquia DOM
- ✗ Mock de tudo — testes desacoplam de realidade
- ✗ `retries: 3` sem investigar root cause de flake
- ✗ CI sem sharding — feedback de horas em suítes grandes
- ✗ Sem trace habilitado — debug demanda reproduzir local

Atividade — Lab Playwright SPA (15 pts)

Entrega: ver Canvas.

Entregar repo com:

1. **Auth state reuse** correto (storageState reutilizado entre tests) — 3pts
2. **Network mocking + HAR replay** — 3pts
3. **Visual regression** em 3 viewports com baseline versionado — 3pts
4. **Trace + report em CI** (artifact uploaded) — 3pts
5. **Sharding 4-way** configurado — 2pts
6. **Padrões aplicados** (POM evoluído ou Fixtures customizadas) — 1pt

| Critério eliminatório: suíte deve passar 100% em 3 runs consecutivos. Flake > 10% = -3pts.

Leituras obrigatórias

- **Microsoft Playwright Docs** — Best Practices (oficial)
- **Winteringham, M.** — *Testing Web APIs* (Manning), cap 5–7
- **Axelrod, A.** — *Complete Guide to Test Automation* (Apress), cap 8–10
- **Google Testing Blog** — *Just Say No to More End-to-End Tests*

Próximo módulo (M3)

Testes de SPA & PWA

- Hydration testing
- Service Worker testing
- Offline mode tests
- Background Sync verification
- Lighthouse CI com budgets

Pré-requisito: Lab Playwright SPA entregue.

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Office hours: quintas 19h